TP2 : entraîner, évaluer et sauvegarder un modèle de sentiment

La démarche pas à pas, étape par étape et test par test

A. Larhlimi

Développement et déploiement d'applications intelligentes

Semaine 2 · TP de 2 h · en binôme

Objectif : un modèle dont on sait ce qu'il vaut

Produire un modèle **reproductible**, **évalué correctement** et **sauvegardé** dans un fichier que l'API de la semaine 3 chargera. Pas un bon modèle : un modèle dont vous savez pourquoi il se trompe.

Ce que la fiche du cas 3 annonçait (TP1)	**Ce que le TP2 en fait**
Approche : machine learning	Un pipeline TF-IDF et régression logistique
Données : étiquetées via la note en étoiles	Un jeu de démonstration, puis 20 000 avis publics
Métrique : rappel sur la classe négative	Mesuré sur un jeu de test, inscrit dans `metrics.json`
Erreur peu grave, requalification d'un clic	Les erreurs sont lues et expliquées, pas seulement comptées

Première séance où le dépôt contient quelque chose qui prédit.

Les cinq étapes, les cinq contrôles

	Durée	Ce que fait le binôme	Contrôle
A	15 min	Installer le kit dans le dépôt du TP1	C1 : 2 failed, 38 passed, 2 skipped
B	15 min	Générer et explorer le jeu de démonstration	C2 : 290 avis, 145 de chaque classe
C	25 min	Entraîner le pipeline, sauvegarder	C3 : F1 macro 0,879 ; metrics.json écrit
D	30 min	Évaluer : matrice de confusion, erreurs	C4 : 7 erreurs, toutes sur des avis mitigés
E	35 min	Vrai jeu Allociné, tests, push	C5 : F1 0,920 ; 42 passed ; CI verte

Prérequis : le dépôt du TP1 à 38 passed. Sinon, on règle cela d'abord.

Étape A. Installer le kit

15 min, à deux

Décompresser le kit **à côté** du dépôt, pas dedans.

```
cd tp2-semaine2
python appliquer_tp2.py ../sentiment-app --simulation    # voir sans écrire
python appliquer_tp2.py ../sentiment-app                 # copier
cd ../sentiment-app && source .venv/bin/activate
python -m pytest -q
```

Contrôle C1 : le point de départ

2 failed, 38 passed, 2 skipped. Les 38 verts sont ceux du TP1, inchangés. Les 2 échecs réclament `models/metrics.json`, les 2 ignorés attendent le modèle : c'est le travail du jour.

Étape B. Le jeu de démonstration

15 min

```
python src/make_demo.py        # 290 avis, graine fixée à 42
```

Famille	Nombre
Nettement positifs	80
Nettement négatifs	80
Mitigés	100
Neutres	30

La forme d'un avis mitigé

« Service impeccable et livraison en avance **sur le papier**, **sauf que ensuite** produit cassé à l'arrivée. »

L'étiquette suit la **seconde** moitié, celle qui conclut.

C2 : 290 avis, 145 de chaque classe. Cette structure explique toutes les erreurs de l'étape D.

Étape C. Entraîner et sauvegarder

25 min

```
python src/train.py
```

```
Source : demo, 290 avis (145 négatifs)
232 avis d'entraînement, 58 de test
Vocabulaire : 316 termes
F1 macro sur le jeu de test : 0.879 (seuil 0.85)
```

Contrôle C3 : deux fichiers, deux destins

`models/metrics.json` apparaît dans `git status`, `models/sentiment.joblib` non, `data/demo_reviews.csv` non plus. Ce qui se régénère par une commande n'entre pas dans Git : seule la **mesure** est versionnée.

Étape C. Les trois lignes à lire

```
X_train, X_test, y_train, y_test = train_test_split(
    df["review"], df["label"], test_size=0.2, random_state=42, stratify=df["
        label"])

Pipeline([("tfidf", TfidfVectorizer(ngram_range=(1, 2), min_df=2)),
          ("clf",   LogisticRegression(max_iter=1000, class_weight="balanced
              "))])
```

Le découpage avant la vectorisation. Apprendre le vocabulaire sur toutes les données ferait fuiter le jeu de test : le score monte, et il est faux.

Le pipeline, un seul objet. Sauvegarder le classifieur sans son vectoriseur donne un modèle inutilisable, et garantit la même transformation en production qu'à l'entraînement.

La graine. Sans `random_state`, deux exécutions donnent deux scores : plus aucune comparaison possible.

Étape D. La matrice de confusion

30 min

```
python src/evaluate.py
```

vérité ↓	nég.	pos.
négatif	**27**	2
positif	5	**24**

7 erreurs sur 58 · exactitude 0,879

- **2 faux positifs** : un client mécontent traité en retard. **Le cas qui compte.**
- **5 faux négatifs** : un avis positif remonté pour rien, sans gravité.

L'exactitude ne dit rien de cela. La fiche du cas 3 visait le **rappel sur la classe négative** : c'est lui qu'on regarde.

Étape D. Pourquoi ces sept erreurs

Toutes les erreurs sont des avis mitigés

« Service impeccable et livraison en avance sur le papier, sauf que ensuite produit cassé à l'arrivée. » vérité : négatif, prédit : positif

Le modèle compte des mots positifs et des mots négatifs, puis tranche en faveur du camp le plus fourni. Il ne voit pas le retournement, parce qu'un sac de mots n'a **pas d'ordre** : « bon au début, mauvais ensuite » et « mauvais au début, bon ensuite » lui donnent le même vecteur.

Ce que vous écrivez

`docs/semaine2.md`, trois phrases : quelles erreurs, pourquoi, et ce qu'il faudrait pour les corriger (des représentations qui tiennent compte de l'ordre : les embeddings, semaine 9).

```
python src/train.py --allocine      # 20 000 avis publics, 7,6 Mo
```

	Démonstration	Allociné
Avis	290	20 000
Vocabulaire	316 termes	149 529 termes
F1 macro	0,879	**0,920**
Rappel sur la classe négative	0,931	0,911

Le modèle n'a pas changé : ce sont les données. Et la mesure devient crédible, calculée sur 4 000 avis au lieu de 58, où une erreur de plus déplaçait le score de deux points.

Étape E. Tests, push, intégration continue

```
python -m pytest -q          # 42 passed = 38 du TP1 + 4 du modèle
git add -A && git commit -m "Semaine 2 : modele, metriques et tests"
git push
```

C5 : ce que vérifie la CI, et comment

La machine d'intégration n'a jamais vu votre modèle : il n'est pas dans le dépôt. Les deux tests de prédiction sont donc ignorés, mais les deux tests de qualité passent : ils lisent `models/metrics.json`, qui est versionné.

Votre CI vérifie qu'un modèle acceptable a été produit, sans transporter le modèle. Premier geste MLOps du semestre.

Les cinq pièges de la séance

Piège	Conséquence	Correction
Vectoriser avant de découper	Score artificiel, effondrement en production	`train_test_split` d'abord
Sauvegarder le classifieur seul	Modèle inutilisable, plus de vocabulaire	Sauvegarder le `Pipeline`
Commiter `sentiment.joblib`	Dépôt qui gonfle à chaque réentraînement	`models/*` ignoré, `metrics.json` versionné
Évaluer sur l'entraînement	Un F1 proche de 1 qui ne veut rien dire	Toujours le jeu de test
Oublier `random_state`	Deux exécutions, deux scores	`random_state=42` partout

Livrables et suite

Sur `main`, en fin de séance

- ☐ `src/` : quatre scripts exécutables
- ☐ `models/metrics.json` versionné, F1 au-dessus de 0,85
- ☐ `sentiment.joblib` produit, jamais commité
- ☐ `pytest -q` affiche 42 passed
- ☐ CI verte
- ☐ `docs/semaine2.md` : trois phrases

Ce que la semaine 3 en fera

La fonction `predict()` de `src/predict.py` sera appelée telle quelle par une API FastAPI. Rien d'autre ne changera : le modèle reste un fichier local, les métriques restent versionnées, et les tests d'aujourd'hui continuent de tourner à chaque push.